

Unit #: 2

Unit Name: Counting, Sorting and Searching

Topic: Programs on Arrays and Strings

Course objectives:

The objective(s) of this course is to make students

CObj1: Acquire knowledge on how to solve relevant and logical problems using computing machine

CObj2: Map algorithmic solutions to relevant features of C programming language constructs

CObj3: Gain knowledge about C constructs and it's associated eco-system

CObj4: Appreciate and gain knowledge about the issues with C Standards and it's respective behaviors

CObj5: Get insights about testing and debugging C Programs

Course outcomes:

At the end of the course, the student will be able to

CO1: Understand and apply algorithmic solutions to counting problems using appropriate C Constructs

CO2: Understand, analyse and apply text processing and string manipulation methods using C Arrays, Pointers and functions

CO3: Understand prioritized scheduling and implement the same using C structures

CO4: Understand and apply sorting techniques using advanced C constructs

CO5: Understand and evaluate portable programming techniques using preprocessor directives and conditional compilation of C Programs

Team – PSWC,

Jan - May, 2022

Dept. of CSE,

PES University

List of Problems

1. Program to find the count of given character in a given string.
2. WAP to input a string and print the Square of all the Numbers in a String. Take care of special cases. Input: wha4ts12ap1p and Output: 16 1 4 1
3. Write a function to find the position of First occurrence of the input character in a given string. Use this function to find the positions of all occurrences of the input character in the string
4. Write a function which takes two string arguments and find the Cartesian product of passed strings.
Input: str1: abc str2: ab and Output: aa, ab, ba, bb, ca, cb
5. WAP to find the sum of all substrings of a string representing a number.
Input : num = "1234"
Output : 1670 Sum = $1+2 + 3 + 4 + 12 + 23 + 34 + 123 + 234 + 1234 = 1670$
6. Program to find the addition of pair of elements in a given array
input: 2 3 4 5 6 7 output: 5 9 13
input: 2 3 4 5 6 output: 5 9 6
7. Write a function to reverse a string using recursion and test the function
8. Write a program to delete the characters from string1 which are present in string2.
input: string1: whatsapp string2: wat output: hspp
9. Write a program to rotate the array based on the rotation value entered by the user
input: 12345 rotation_val: 2 output: 34512
10. Write a program to find if two strings are anagrams.
Anagrams: Same set of letters with different arrangements. Number of times a letter present in one string must be equal to the number of times the letter is present on another string
11. Given a number n, WAP to find all 1 to n bit numbers with no consecutive 1s in their binary representation.
Input : n = 4 Output : 1 2 4 5 8 9 1 These are numbers with 1 to 4 bits and no consecutive ones in binary representation.
Input : n = 3 Output : 1 2 4 5

A few Solutions

1. Program to find the count of given character in a given string.

Solution:

```
int main()
{
    char str1[100]; printf("enter the string\n");
    scanf("%[^\\n]s",str1);    // To receive multiple words and store multiple words under one name
    printf("enter the character\n");
    fflush(stdin);
    char ch = getchar();
    int count = find_frequency(str1, ch);
    printf("%c is present in %s, %d times\n",ch, str1,count);
    return 0;
}

int find_frequency(char *s, char ch)
{
    int count = 0;
    for(; *s ; s++)           // exit the loop when *s is '\\0'
    {
        if (*s == ch) count++;
    }
    return count;
}
```

2. WAP to input a string and print the Square of all the Numbers in a String. Take care of special cases. Input: wha4ts12ap1p and Output: 16 1 4 1

Solution:

```
#include<stdio.h>

int sqr(int r);
void extract_num_square(char *c);
int main()
{
    char ch[400];
    scanf("%s",ch); //wha4ts12ap1p
    extract_num_square(ch);
    return 0;
}

int sqr(int r)
{
    return r*r;
}

void extract_num_square(char *c)
{
    while(*c != '\0')
    {
        if((*c >= '1' && *c <= '9'))
        {
            printf("%c--",(*c));
            printf("%d\n",sqr(*c-'0'));
        }
        c++;
    }
}
```

3. Write a function to find the position of First occurrence of the input character in a given string. Use this function to find the positions of all occurrences of the input character in the string.

Solution:

```
#include<stdio.h>

int find_first(char* ,char);

int main()
{
    char text[100];
    printf("enter the text\n");
    scanf("%[^\n]s",text);        // Till \n, receive input and store in one char array variable
    fflush(stdin);
    printf("enter the character which u want to find\n");
    char ch = getchar();
    int res = find_first(text,ch);
    if(res == -1)
        printf("not present in the text\n");
    else
        printf("available at %d position\n",res);
    return 0;
}
```

Different versions of find_first function are as below.

Version 1:

```
int find_first(char *a,char c)
{
    int p = -1;
    int i;
    int check = 0;
    for(i = 0;i<strlen(a);++i)
    {
        if(a[i]==c && check == 0)
        {
```

```
        p = i; check = 1;
    }
}
return p;
}
```

Version 2:

```
int find_first(char *a, char c)
{
    int i = 0;
    while((a[i]) && a[i] != ch )
    {
        i++;
    }
    return a[i]? i: -1;
}
```

Version 3:

```
int find_first(char *a,char ch)
{
    int i = 0;
    while(*a && *a != ch )
    {
        a++;i++;
    }
    return (*a) ? i : -1;
}
```

Use this solution to find the count of given character in a string.

Think: Can you use recursive function for this?

Part 2: Using this function to find all occurrences of a character.

```
int main()
{
    char str[100];
    printf("enter the string\n");
    scanf("%s",str);
    fflush(stdin);
    char ch;
    printf("enter the character\n");
    ch = getchar();
    //find all occurrences
    int pos = -1;
    int i;
    while( (i = find_first(str + pos + 1, ch)) != -1)
    {
        pos = pos + i + 1;
        printf("%d ", pos);
    }
    printf("\n");
    return 0;
}
```

6. Program to find the addition of pair of elements in a given array

input: 2 3 4 5 6 7 output: 5 9 13

input: 2 3 4 5 6 output: 5 9 6

Solution:

```
#include<stdio.h>

void pair_addition(int *a, int n);

int main()
{
    int a1[] = {2,5,6,7,8,1};
    int a2[] = {2,5,6,7,8};
    int n1 = sizeof(a1)/sizeof(*a1);
```

```
    int n2 = sizeof(a2)/sizeof(*a2);  
    pair_addition(a1,n1);  
    pair_addition(a2,n2);  
    return 0;  
}  
void pair_addition(int *a, int n)  
{  
    int i;  
    for(i = 0; i < n-1; i += 2)  
    {  
        int sum = a[i]+a[i+1];  
        printf("%d\t",sum);  
    }  
    if(n%2)  
        printf("%d",a[i]);  
    printf("\n");  
}
```

Happy Coding!!